Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

Лабораторная работа №1

По дисциплине Современные методы искусственного интеллекта

Тема работы Классификация изображений с применением transfer learning

Обучающийся Сакулин Иван Михайлович

Факультет Факультет прикладной информатики

Группа K3321

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся	_____	_____	Сакулин И.М.
	(дата)	(подпись)	(Ф.И.О.)
Руководитель	_____	_____	Царёв Д.В.
	(дата)	(подпись)	(Ф.И.О.)

2026

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ **4**

1 БАЗОВЫЕ МЕТОДЫ МАШИННОГО ОБУЧЕНИЯ **5**

1.1 Предварительная обработка данных 5

1.1.1 Бинаризация 5

1.1.2 Исключение среднего 6

1.1.3 Масштабирование 7

1.1.4 Нормализация 7

1.2 Кодирование меток 8

1.3 Логистический классификатор 10

1.4 Наивный байесовский классификатор 11

1.5 Матрица неточностей 13

1.6 Машины опорных векторов 14

1.7 Регрессия 17

1.7.1 Одномерный регрессор 17

1.7.2 Многомерный регрессор 18

1.8 Оценка стоимости недвижимости с помощью SVR 19

2 ИСПОЛЬЗОВАНИЕ ПРЕДОБУЧЕННЫХ НЕЙРОСЕТЕЙ ... **22**

2.1 Выбор датасета Oxford-IIIT Pet 22

2.2 Выбор архитектуры ResNet18 22

2.3 Подготовка данных 23

2.3.1 Предварительная обработка изображений 23

2.3.2 Разделение датасета 24

2.3.3 Загрузка данных 24

2.4 Feature Extraction 25

2.4.1 Настройка модели 25

2.4.2 Обучение модели 25

2.4.3 Оценка модели 27

2.5 Fine-tuning 28

2.5.1 Настройка модели 28

2.5.2 Обучение модели 29

2.5.3 Оценка модели 31

2.6 Сравнение 32

ЗАКЛЮЧЕНИЕ 34

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 35

ВВЕДЕНИЕ

Цель работы — получить практические навыки по обучению распознавания образов, кластеризации и классификации данных с помощью обучения с учителем и метода transfer learning.

Задачи:

- освоить методы предварительной обработки данных средствами `sklearn`;
- изучить кодирование меток;
- построить логистический классификатор и исследовать влияние параметра регуляризации;
- обучить наивный байесовский классификатор и оценить его качество;
- построить матрицу неточностей и проанализировать отчёт о классификации;
- обучить классификатор на опорных векторах на наборе `income_data`;
- построить одномерный и многомерный регрессоры;
- построить SVR-регрессор для оценки стоимости жилья и исследовать влияние гиперпараметров;
- оформить отчёт в соответствии с требованиями.

Задачи дополнительной части:

- выбрать датасет для задачи классификации изображений (не менее трёх классов);
- выбрать предобученную архитектуру ResNet, EfficientNet или MobileNet и обосновать выбор;
- реализовать стратегию Feature Extraction: заморозить веса базовой сети, заменить выходной слой и обучить модель;
- оценить качество классификации и построить матрицу ошибок;
- реализовать стратегию Fine-tuning: разморозить часть верхних слоёв базовой сети и провести дообучение с низким темпом обучения;
- сравнить результаты двух подходов по метрикам Accuracy, Precision и Recall и сделать вывод о целесообразности дообучения весов базовой сети.

Все программы, обсуждаемые в работе, доступны в репозитории [1].

1 БАЗОВЫЕ МЕТОДЫ МАШИННОГО ОБУЧЕНИЯ

1.1 Предварительная обработка данных

Входная выборка задана массивом 4×3 (рисунок 1).

```
input_data = np.array([
    [ 5.1, -2.9,  3.3],
    [-1.2,  7.8, -6.1],
    [ 3.9,  0.4,  2.1],
    [ 7.3, -9.9, -4.5]
])
```

Рисунок 1 — Входная выборка 4×3

1.1.1 Бинаризация

Бинаризация выполнялась методом `preprocessing.Binarizer` с порогом 2.1. Результат приведён на рисунке 2.

```
# Бинаризация данных
data_binarized = preprocessing.Binarizer(threshold=2.1).transform(input_data)
print("\nOriginal data:\n", input_data)
print("\nBinarized data:\n", data_binarized)
```

✓ 0.0s

```
Original data:
 [[ 5.1 -2.9  3.3]
 [-1.2  7.8 -6.1]
 [ 3.9  0.4  2.1]
 [ 7.3 -9.9 -4.5]]

Binarized data:
 [[1. 0. 1.]
 [0. 1. 0.]
 [1. 0. 0.]
 [1. 0. 0.]]
```

Рисунок 2 — Результат бинаризации

Сопоставляя исходный массив и вывод на рисунке 2, видно, что все значения строго больше 2.1 заменены на 1, а все остальные — на 0. Бинаризация полезна, когда признак имеет смысл «есть/нет», и не подходит, если важна величина отклонения от порога.

1.1.2 Исключение среднего

Среднее и стандартное отклонение вычислялись до и после применения `preprocessing.scale`. Результат приведён на рисунке 3.

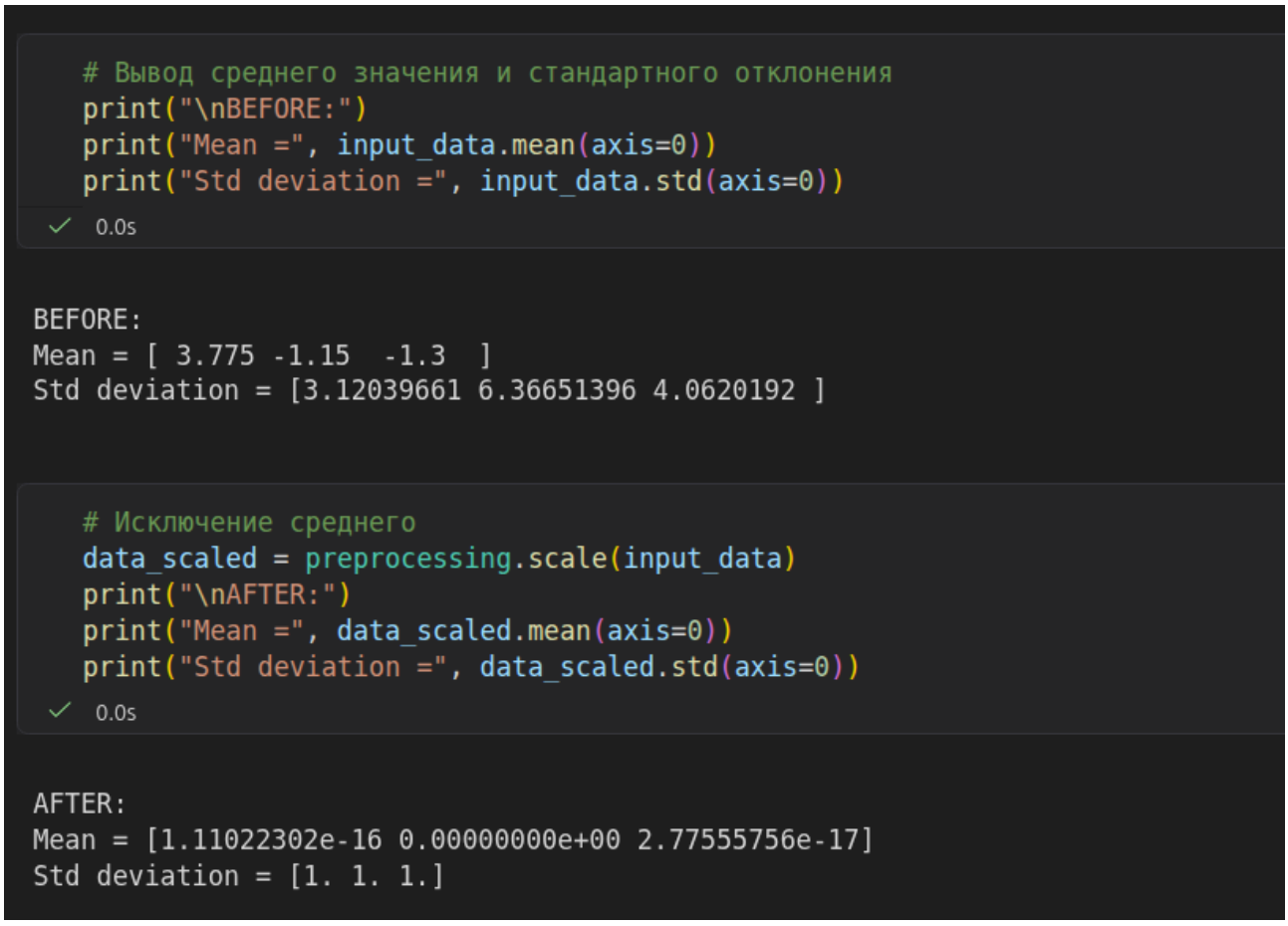


Рисунок 3 — Исключение среднего

До преобразования средние по столбцам сильно отличались от нуля, стандартные отклонения — порядка 3–7. После преобразования, как видно на рисунке 3, средние стали близки к нулю, а стандартные отклонения вовсе стали единицами. Это означает, что каждый признак центрирован и приведён

к одной шкале, поэтому смещение значений больше не влияет на обучение. Обычно эту операцию выполняют перед обучением линейных моделей и SVM.

1.1.3 Масштабирование

Масштабирование выполнялось методом `preprocessing.MinMaxScaler` с диапазоном $(0, 1)$. Результат приведён на рисунке 4.

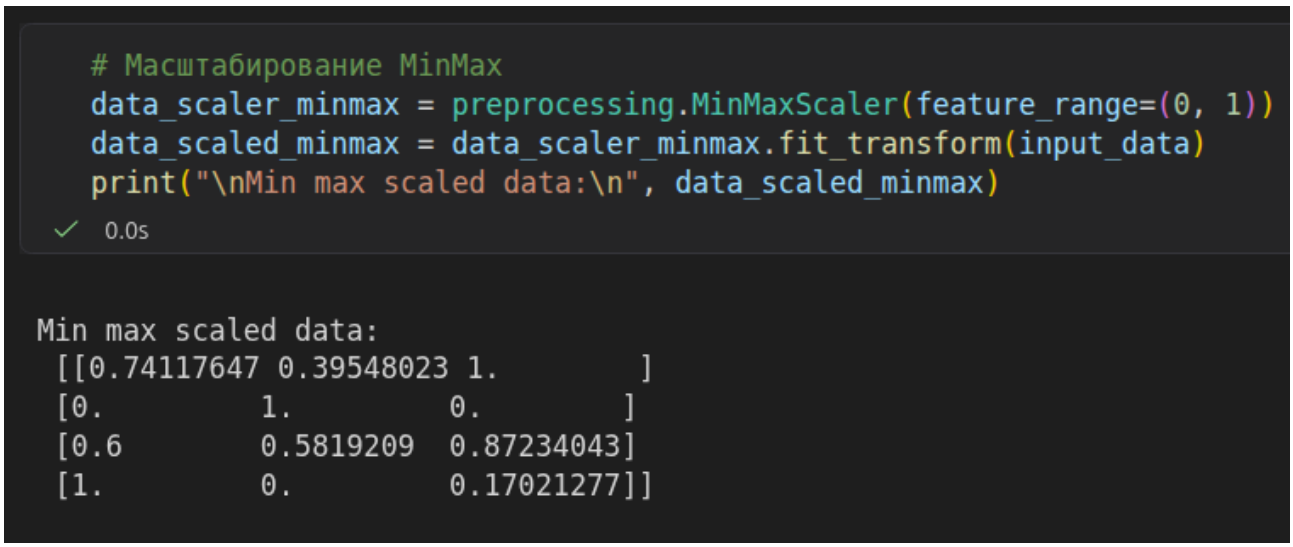


Рисунок 4 — Масштабирование Min–Max

По рисунку 4 видно, что максимальное значение в каждом столбце равно 1, минимальное — 0, а остальные значения линейно отображены в диапазон $[0, 1]$. Такое преобразование полезно, когда признаки измерены в разных единицах, но нужно сохранить форму распределения.

1.1.4 Нормализация

Применены L1 и L2 нормализации методом `preprocessing.normalize`. Результат приведён на рисунке 5.

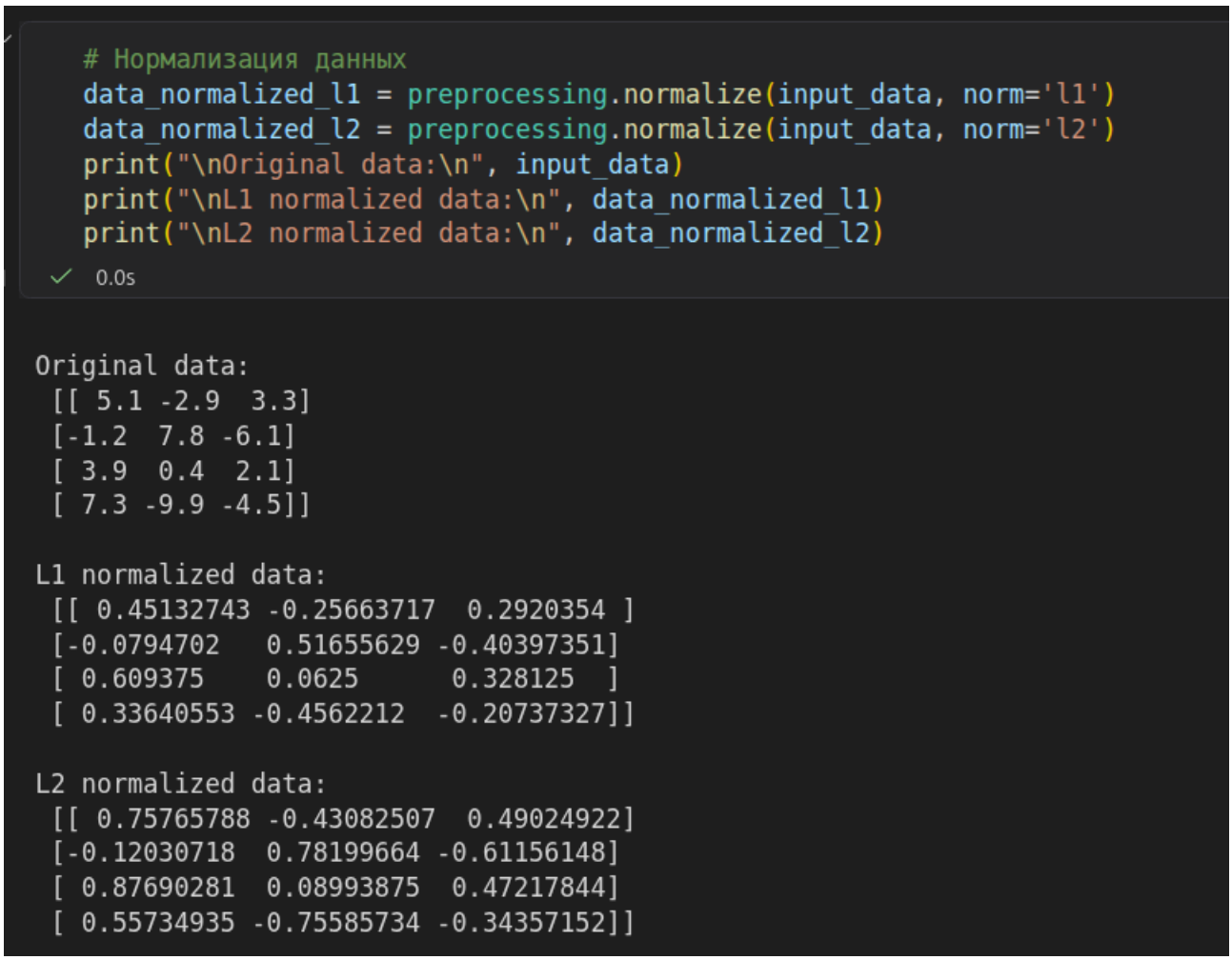


Рисунок 5 — Нормализация L1 и L2

По рисунку 5 можно проверить: для L1 сумма модулей элементов каждой строки равна 1, для L2 сумма квадратов равна 1. Нормализация отличается от масштабирования тем, что она работает со строками (объектами), а не со столбцами (признаками). L1 устойчивее к выбросам, поскольку линейно учитывает вклад каждого значения; L2 сильнее реагирует на большие значения, поэтому её применяют, когда выбросы важны.

1.2 Кодирование меток

Метки преобразовывались в числовой вид через `LabelEncoder`, после чего выполнялось обратное преобразование. Использовался следующий набор меток: `['red', 'black', 'red', 'green', 'black', 'yellow', 'white']`. Результат приведён на рисунке 6.

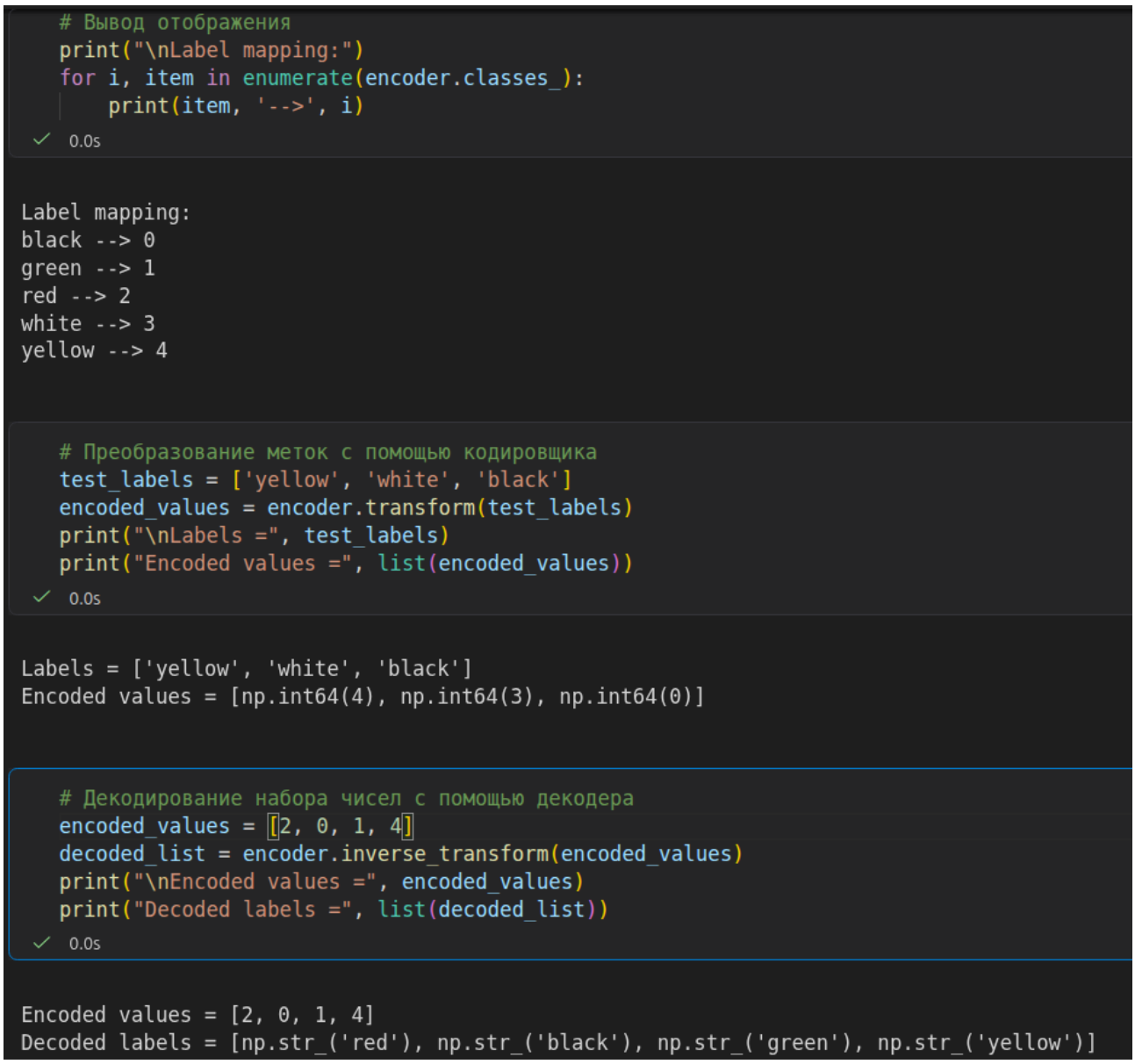


Рисунок 6 — Кодирование и декодирование меток

Как показано на рисунке 6, кодировщик упорядочил уникальные метки по алфавиту и присвоил им индексы. Проверочный набор дал значения, что совпадает с отображением. Обратное преобразование вектора вернуло метки.

Кодирование меток один из способов обработки строковых данных. Еще один способ кодирования меток для несвязанных категорий `one-hot encoding` (с курса машинного обучения) нужен, чтобы модель не воспринимала классы как упорядоченные.

1.3 Логистический классификатор

Логистическая регрессия обучалась на двумерных векторах, соответствующих четырём классам (рисунок 7).

```
X = np.array([
    [3.1, 7.2], [4, 6.7], [2.9, 8], [5.1, 4.5], [6, 5], [5.6, 5],
    [3.3, 0.4], [3.9, 0.9], [2.8, 1], [0.5, 3.4], [1, 4], [0.6, 4.9]
])
y = np.array([0, 0, 0, 1, 1, 1, 2, 2, 2, 3, 3, 3])
```

Рисунок 7 — Входная выборка 4×3

Использовался `LogisticRegression` с решателем `liblinear`. Для анализа изменялся параметр C - параметр обратной регуляризации (обратная штрафу за ошибку величина).

Результат при $C = 1$ приведён на рисунке 8.

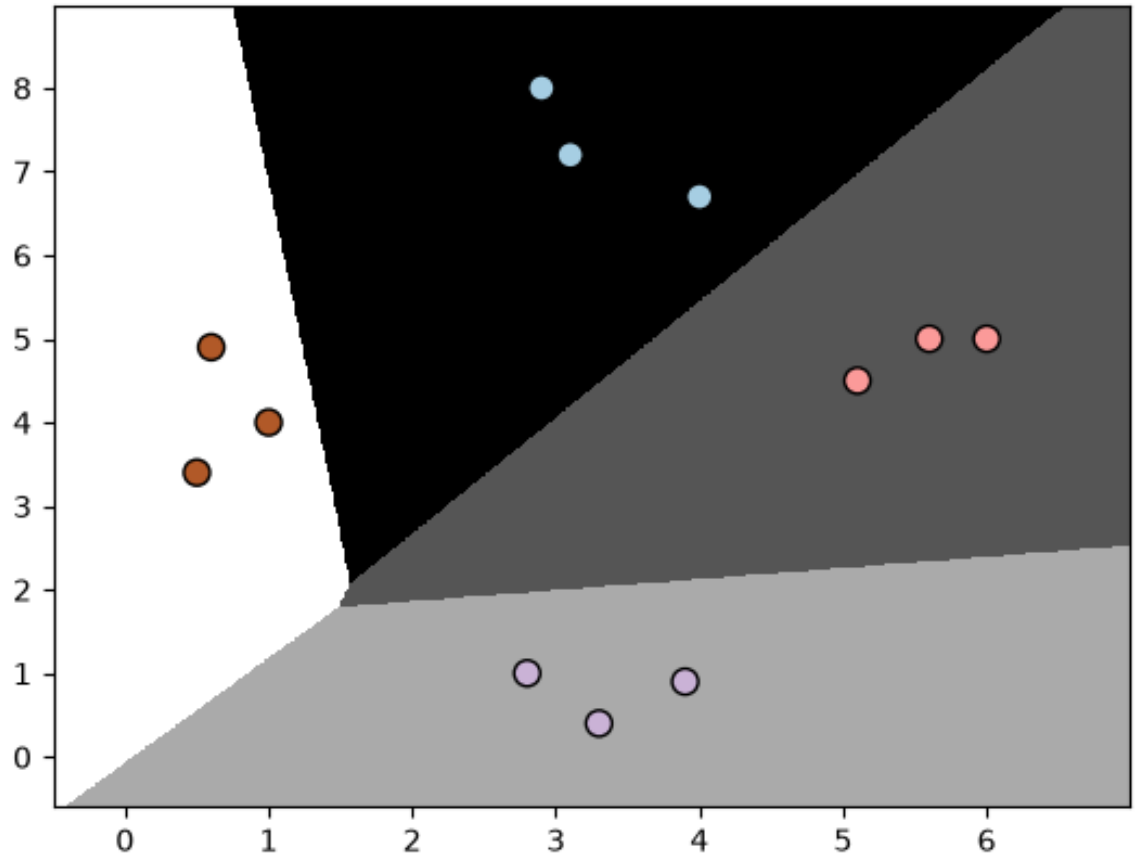


Рисунок 8 — Логистический классификатор при $C = 1$

Результат при $C = 100$ приведён на рисунке 9.

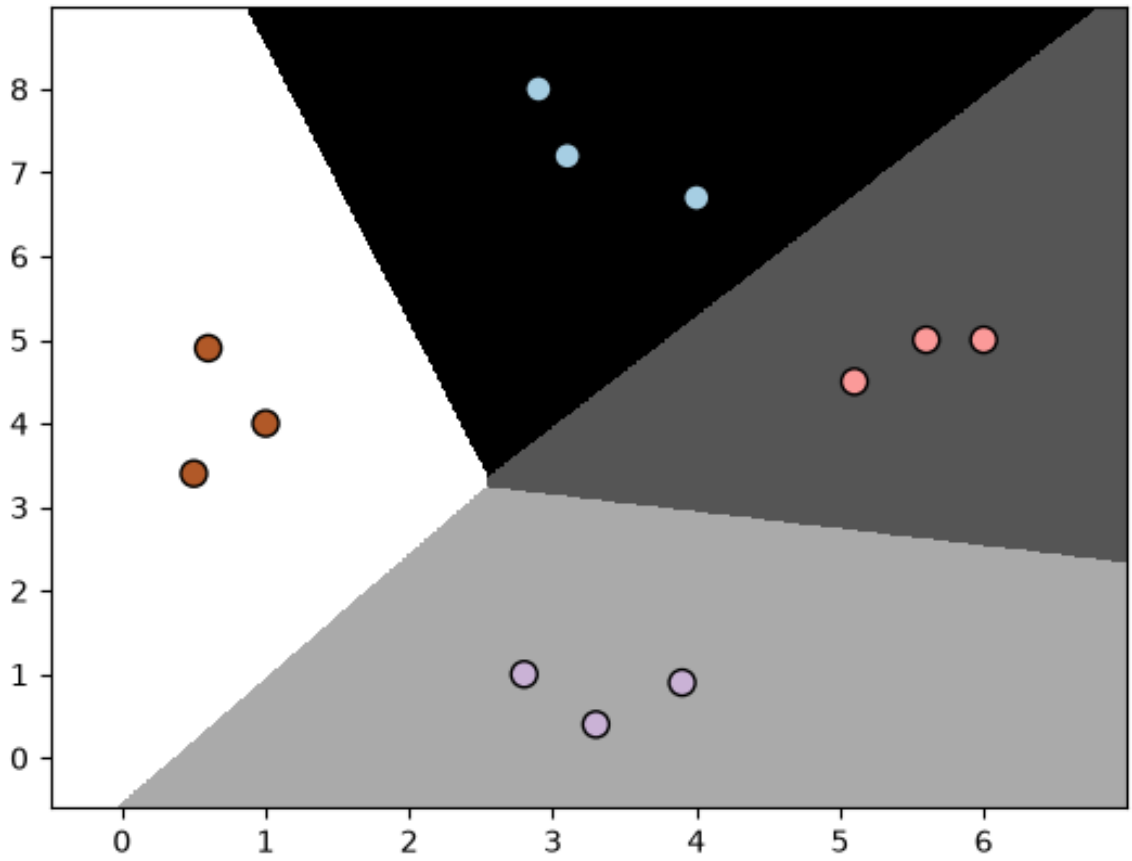


Рисунок 9 — Логистический классификатор при $C = 100$

При увеличении C со 1 до 100 штраф за ошибки классификации возрастает, и модель сильнее подстраивается под обучающие данные. При малом $C = 1$ могло произойти недообучение, на рисунке 8 видно, что границы классов прижимаются к красным и фиолетовым точкам ближе к нулю. При большом $C = 100$ границы стали выглядеть, они равноудалены от точек разных классов. Однако, если задать слишком большой C модель может переобучиться.

1.4 Наивный байесовский классификатор

Использовался `GaussianNB` на данных `data_multivar_nb.txt`.

На рисунке 10 классы разделены корректно, ошибочных точек нет.

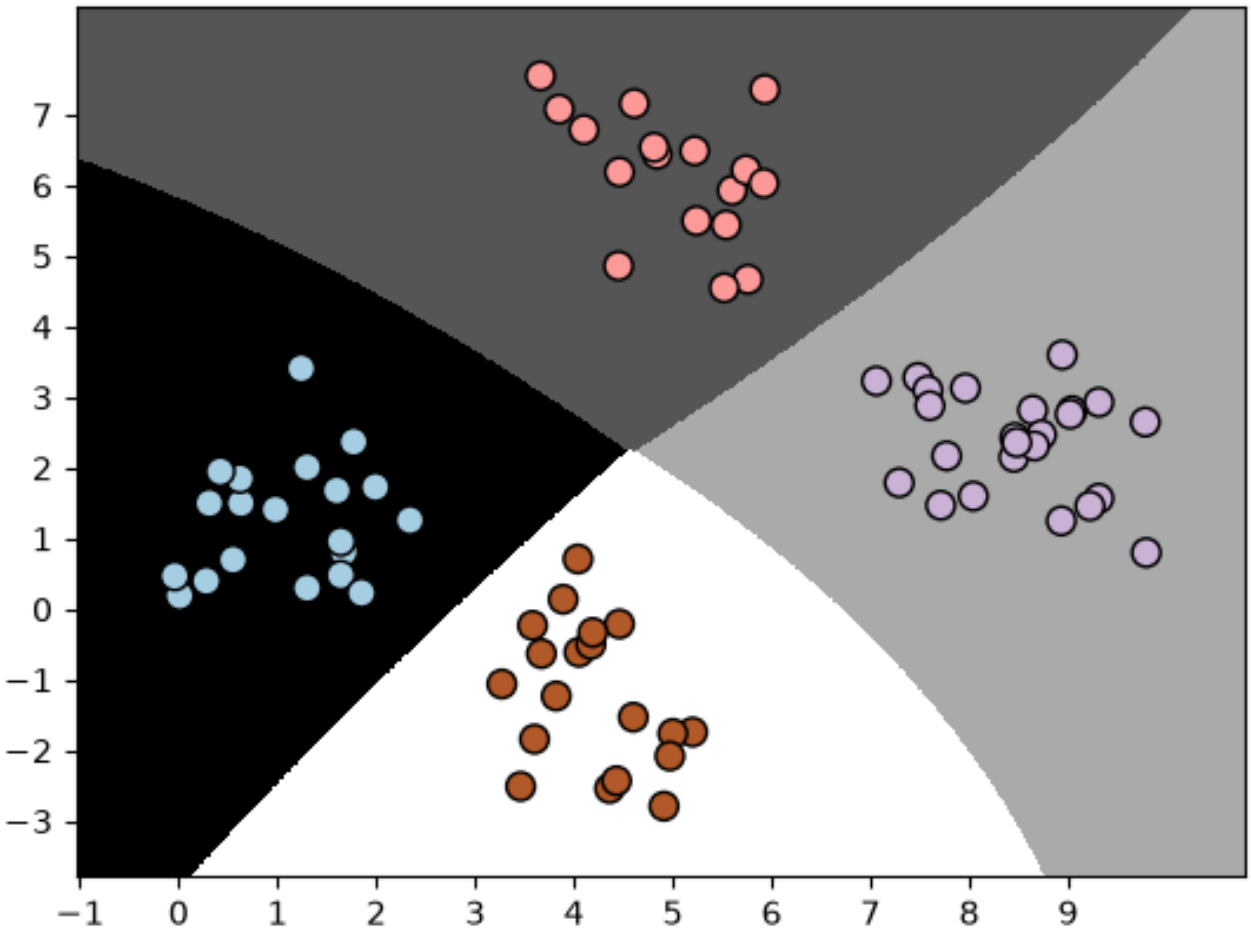


Рисунок 10 — Разбиение данных наивным байесовским классификатором

Метрики на рисунке 11 близки к 100%, precision и recall совпадают — модель сбалансирована и не смещена в сторону одного класса.

```
Оценка показателей классификатора
Accuracy: 99.75%
Precision: 99.76%
Recall: 99.75%
F1: 99.75%
```

Рисунок 11 — Ключевые показатели классификатора

Кросс-валидация даёт более объективную оценку, чем единственное разбиение, поскольку каждый объект хотя бы раз попадает в тестовую выборку.

1.5 Матрица неточностей

Истинные метки: `[2,0,0,2,4,4,1,0,3,3,3]`, предсказанные: `[2,1,0,2,4,3,1,0,1,3,3]`. Результаты приведены на рисунках 12 и 13.

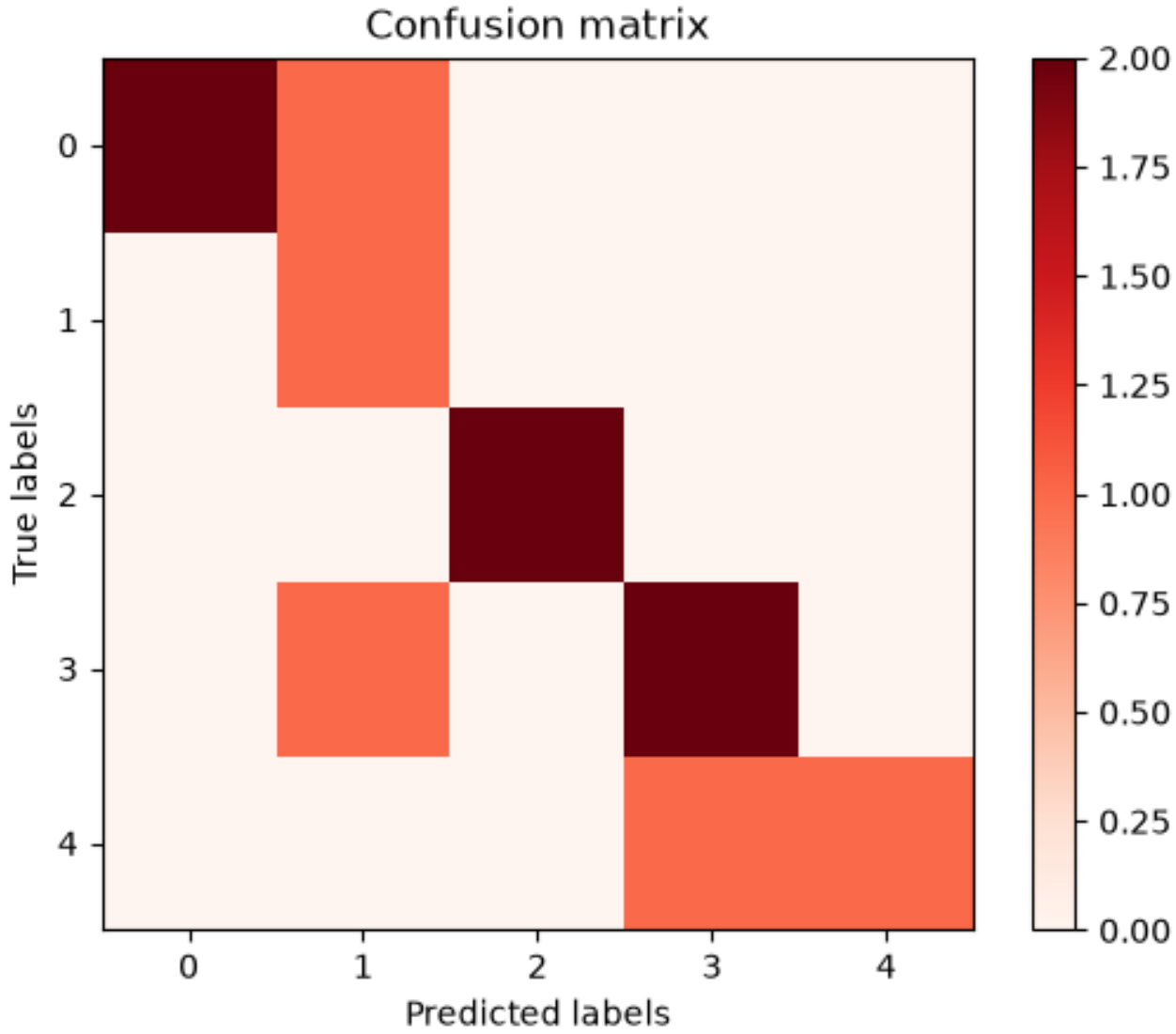


Рисунок 12 — Матрица неточностей

По рисунку 12 матрица имеет размер 5×5, так как мы имеем всего 5 классов. На главной диагонали отображаются корректно предсказанные классы. Все, что больше 0 вне диагонали - ошибки. На текущих данных три ошибки: истинный 0, но предсказано 1, истинный 3, но предсказано 1, истинный 4, но предсказано 3.

```
# Отчет о результатах классификации
targets = ['Class-0', 'Class-1', 'Class-2', 'Class-3', 'Class-4']
print('\n', classification_report(
    true_labels, pred_labels, target_names=targets
))
```

✓ 0.0s

```
              precision    recall  f1-score   support

     Class-0       1.00      0.67      0.80         3
     Class-1       0.33      1.00      0.50         1
     Class-2       1.00      1.00      1.00         2
     Class-3       0.67      0.67      0.67         3
     Class-4       1.00      0.50      0.67         2

    accuracy                           0.73        11
   macro avg       0.80      0.77      0.73        11
weighted avg       0.85      0.73      0.75        11
```

Рисунок 13 — Отчёт о классификации

По рисунку 13 точность (precision) максимальная у классов 0, 2 и 4 (ни одной лишней точки), к классу 1 отнесли две точки не из этого класса, в класс 3 отнесли одну точку не из класса 3. Полнота (recall) максимальная у классов 1 и 2 (все точки найдены), для классов 0 и 3 не было найдено одной истинной точки, для класса 4 не было найдено половины всех точек.

1.6 Машины опорных векторов

Использовался `OneVsOneClassifier(LinearSVC)` на наборе `income_data.txt` для предсказания уровня дохода ($\leq 50K$ или $> 50K$).

На всех данных F1-мера составляет около 76.08%. Это значение можно оправдать тем, что классы несбалансированы (доходов $\leq 50K$ в датасете заметно больше), поэтому линейный SVM не может идеально разделить их в исходном признаковом пространстве.

Далее классификатор проверялся на четырёх точках с разными ядрами и значениями C (рисунок 14).

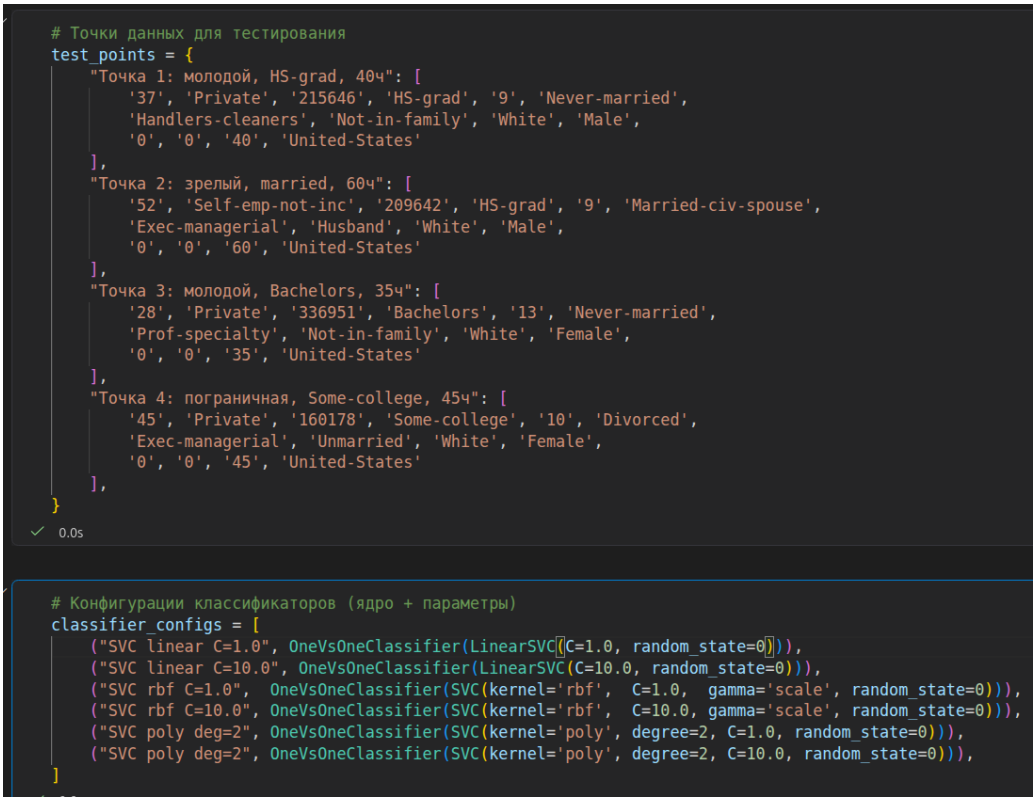


Рисунок 14 — Выбранные точки и ядра

На рисунках 15-17 представлены результаты работы с различными ядрами и точками.

```
================================================================================
Классификатор: SVC linear C=1.0
================================================================================
F1        : 75.75%
Precision : 79.26%
Recall    : 79.56%
F1 (cv=3) : 76.08%
  Точка 1: молодой, HS-grad, 40ч  ->  <=50K
  Точка 2: зрелый, married, 60ч  ->  <=50K
  Точка 3: молодой, Bachelors, 35ч  ->  <=50K
  Точка 4: пограничная, Some-college, 45ч  ->  <=50K

================================================================================
Классификатор: SVC linear C=10.0
================================================================================
F1        : 75.75%
Precision : 79.26%
Recall    : 79.56%
F1 (cv=3) : 76.12%
  Точка 1: молодой, HS-grad, 40ч  ->  <=50K
  Точка 2: зрелый, married, 60ч  ->  <=50K
  Точка 3: молодой, Bachelors, 35ч  ->  <=50K
  Точка 4: пограничная, Some-college, 45ч  ->  <=50K
```

Рисунок 15 — Метрики и прогнозы для линейного ядра

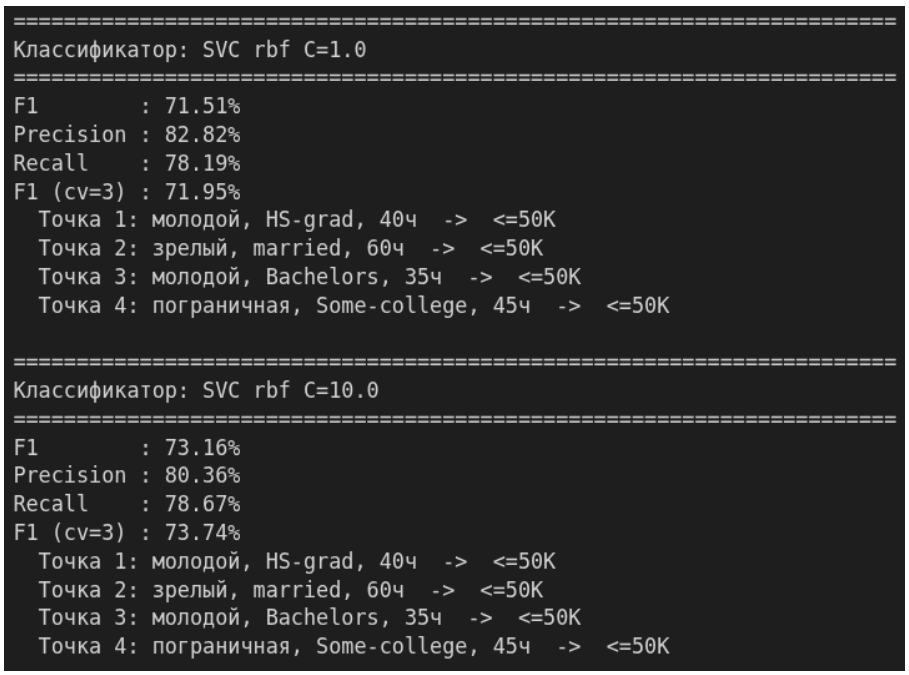


Рисунок 16 — Метрики и прогнозы для радиального базисного ядра

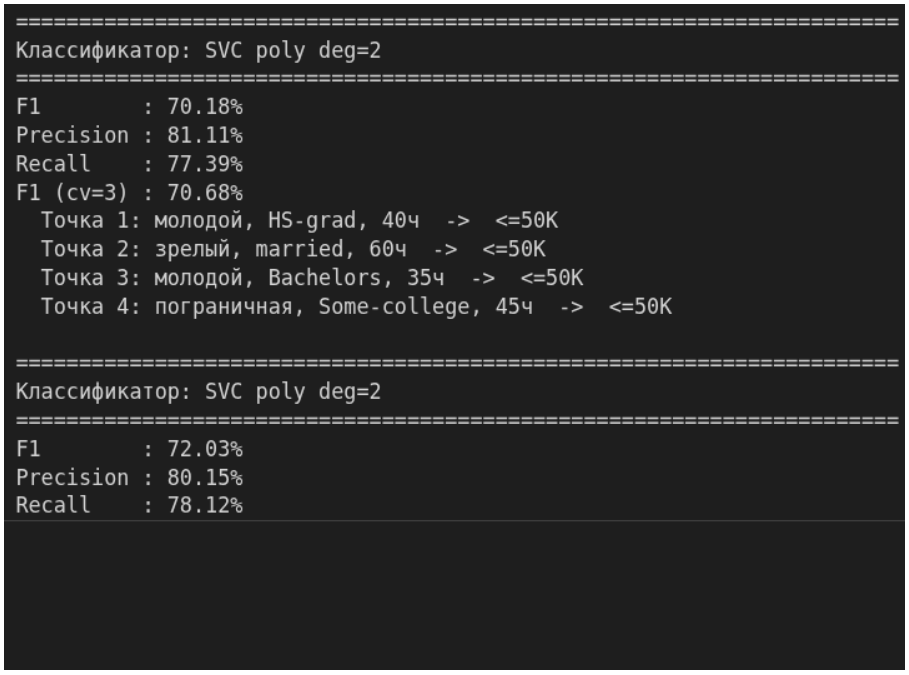


Рисунок 17 — Метрики и прогнозы для полиномиального ядра

Полученные результаты показывают, что линейное ядро дало наилучший результат среди рассмотренных вариантов. При $C = 1$ значение F1-меры составило 75.75%, а при $C = 10$ практически не изменилось и составило 75.75%. При этом значение F1 по кросс-валидации немного увеличилось

с 76.08% до 76.12%, поэтому изменение параметра C в данном диапазоне практически не повлияло на качество модели.

Использование радиального базисного ядра привело к снижению F1-меры до 71.51% при $C = 1$ и до 73.16% при $C = 10$. Полиномиальное ядро также показало более низкий результат: F1-мера составила около $70 - 72\%$. Таким образом, для данного набора данных линейная модель оказалась более подходящей, чем рассмотренные нелинейные ядра.

Для всех проверенных моделей четыре тестовые точки были отнесены к классу $\leq 50K$. При совершенно разных выбранных точках получился одинаковый результат, это объясняется тем, что датасет сильно ассиметричен в сторону $\leq 50K$.

1.7 Регрессия

1.7.1 Одномерный регрессор

На данных `data_singlevar_regr.txt` обучался `LinearRegression`.

На рисунке 19 видно, что облако точек хорошо укладывается на прямую, то есть зависимость близка к линейной. Метрики на рисунке 18 подтверждают это: MAE (среднее отклонение) порядка 3, MSE (средний квадрат отклонения) порядка 15. R^2 около 0.9, близкое к 1, означает, что линейная модель объясняет большую часть дисперсии целевой переменной.

```
Linear regressor performance:
Mean absolute error = 0.59
Mean squared error = 0.49
Median absolute error = 0.51
Explain variance score = 0.86
R2 score = 0.86
```

Рисунок 18 — Метрики одномерного регрессора

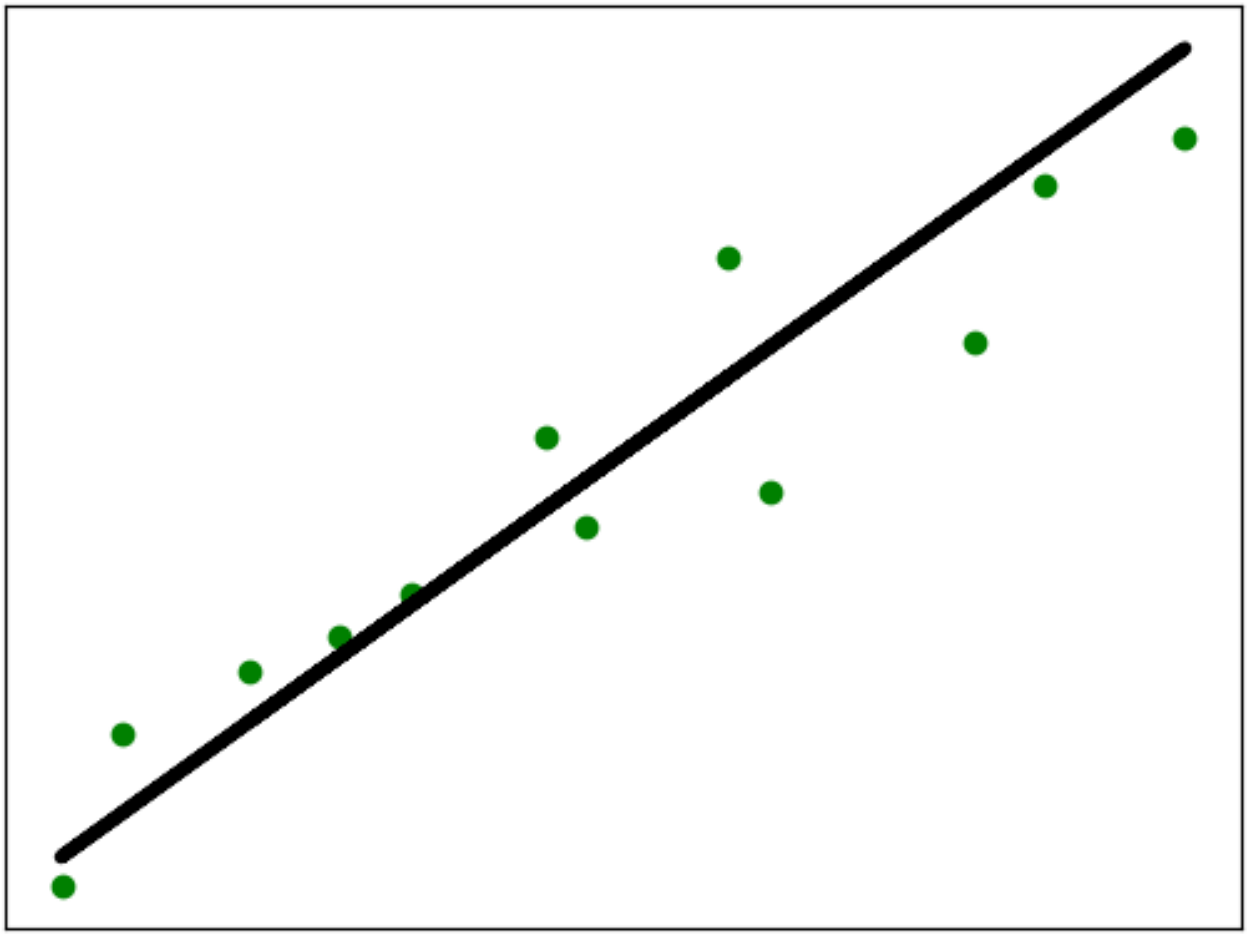


Рисунок 19 — Аппроксимация данных линейным регрессором

1.7.2 Многомерный регрессор

На рисунке 20 представлены результаты оценки линейного регрессора, которые показывают достаточно высокую точность модели и хорошее соответствие предсказанных значений исходным данным.

```
Linear Regressor performance:
Mean absolute error = 3.58
Mean squared error = 20.31
Median absolute error = 2.99
Explained variance score = 0.86
R2 score = 0.86
```

Рисунок 20 — Метрики линейного регрессора

На рисунке 21 представлены результаты оценки полиномиального регрессора, которые свидетельствуют о существенном ухудшении качества прогнозирования по сравнению с линейной моделью.

```
Linear Regressor performance:
Mean absolute error = 11.19
Mean squared error = 907.66
Median absolute error = 5.09
Explained variance score = -4.9
R2 score = -5.04
```

Рисунок 21 — Метрики полиномиального регрессора

На данных `data_multivar_regr.txt` сравнивались линейный регрессор и полиномиальный степени 10. Результат приведён на рисунке **??**.

```
Linear regression:
 [36.05286276]

Polynomial regression:
 [31.63944054]
```

Рисунок 22 — Линейный и полиномиальный многомерные регрессоры

Таким образом, в данном эксперименте линейный регрессор показал более высокое качество на тестовой выборке. Увеличение степени модели до 10 не привело к улучшению результатов: значения ошибок полиномиального регрессора существенно увеличились, а коэффициент R^2 стал отрицательным.

1.8 Оценка стоимости недвижимости с помощью SVR

Для построения регрессионной модели использовался набор данных `California Housing`, содержащий информацию о характеристиках жилых районов Калифорнии. Целевой переменной является медианная стоимость жилья. Были обучены семь моделей `SVR`. Рассматривались линейное и радиально-базисное (`RBF`) ядра с различными значениями параметров C и ϵ. Полученные результаты приведены на рисунке 23.

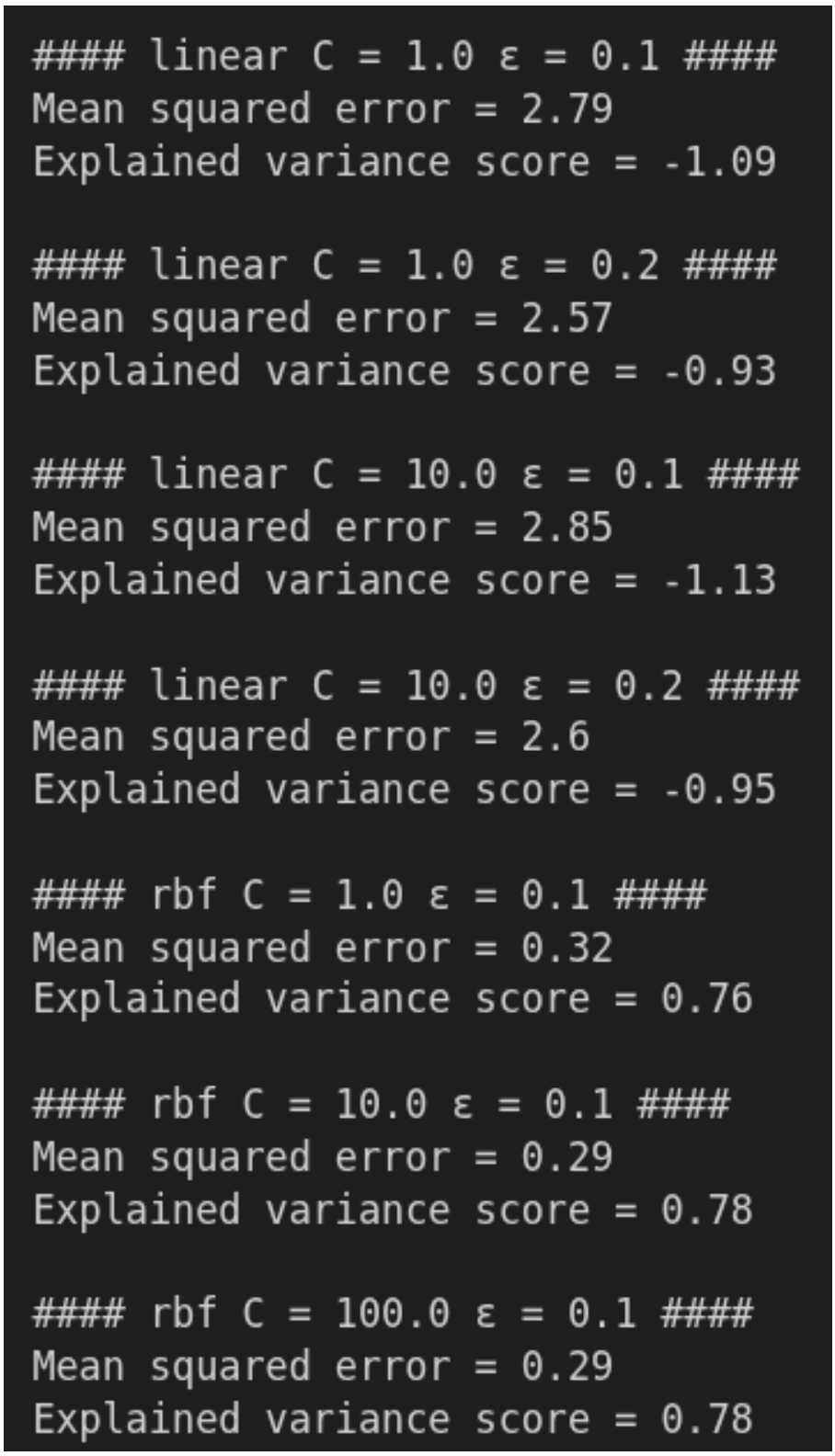


Рисунок 23 — Метрики моделей SVR с различными гиперпараметрами

Модели с линейным ядром показали существенно худшие результаты. Изменение C с 1 до 10 практически не улучшило результаты. Среди рассмотренных моделей наилучшие результаты показали модели с RBF-ядром при

$C = 10$ и $C = 100$. При этом увеличение C с 10 до 100 практически не повлияло на качество модели, поэтому использование $C = 10$ является более рациональным: больше шанс избежать переобучения.

Для дополнительного анализа были выбраны пять тестовых точек, соответствующих различным характеристикам жилых районов. Результаты прогнозирования приведены на рисунке 24.

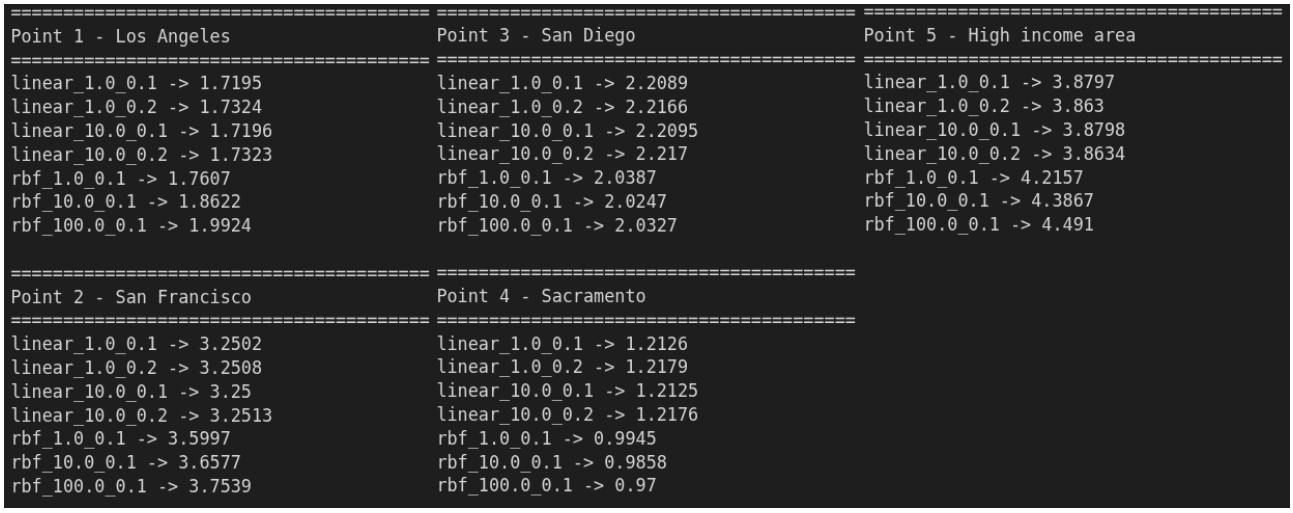


Рисунок 24 — Прогнозы стоимости жилья для тестовых точек

Сравнение результатов показывает, что RBF-модель лучше учитывает нелинейные зависимости между характеристиками района и стоимостью недвижимости. Особенно заметна разница между линейным и RBF-ядром при прогнозировании точек с различными характеристиками. Как и в предыдущем опыте увеличение параметра C в RBF-модели с 1 до 10 заметно улучшило результат, а дальнейшее увеличение до 100 практически не дало прироста качества.

2 ИСПОЛЬЗОВАНИЕ ПРЕДОБУЧЕННЫХ НЕЙРОСЕТЕЙ

2.1 Выбор датасета Oxford-IIIT Pet

Для работы был выбран датасет `Oxford-IIIT Pet`. Датасет содержит изображения домашних животных, распределённые по 37 классам, соответствующим различным породам кошек и собак. Выбор обусловлен личными предпочтениями автора и разнообразием изображений: объекты имеют различный фон, положение и масштаб [2]. Пример изображений на рисунке 25.

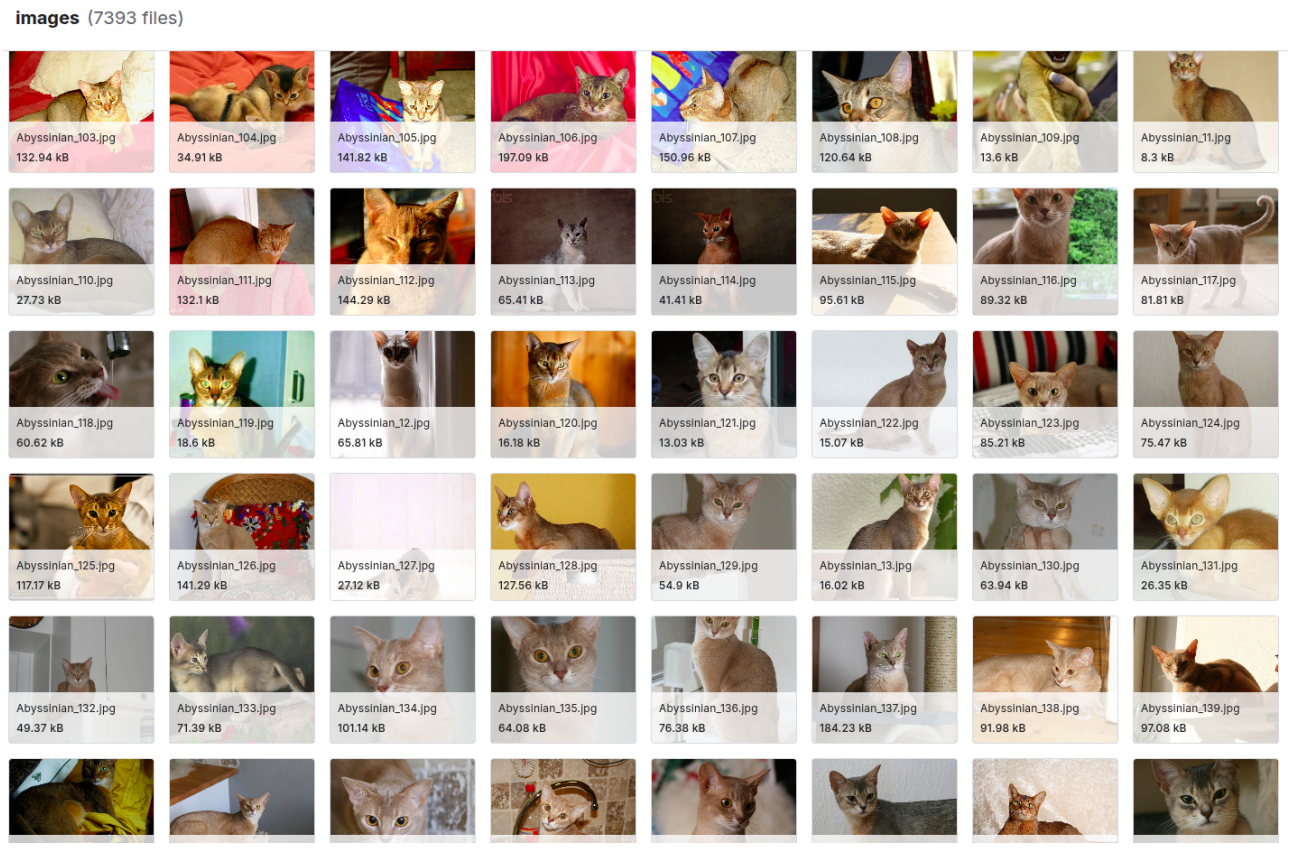


Рисунок 25 — Примеры изображений из датасета Oxford-IIIT Pet

2.2 Выбор архитектуры ResNet18

ResNet выбрана как компромисс между качеством классификации и вычислительной сложностью: основное преимущество MobileNet – работа в ограниченных ресурсах, EfficientNet может быть точнее, но имеет более сложную архитектуру.

ResNet является сверточной нейросетью для классификации изображений, 18 версия была выбрана для уменьшения необходимых вычислительных мощностей. Веса модели предварительно обучены на ImageNet, а последний классификационный слой заменяется на слой с 37 выходами.

2.3 Подготовка данных

2.3.1 Предварительная обработка изображений

Для всех изображений выполняются следующие преобразования, которые необходимы для соответствия объектов датасета объектам формату входных данных предобученной ResNet18:

- изменение размера изображения до 224×224 пикселей;
- нормализация цветовых каналов;
- преобразование изображения в тензор.

Чтобы увеличить разнообразие обучающих данных и улучшить обобщающую способность модели, для тренировочной выборки дополнительно применяются следующие аугментации, пример на рисунке 26:

- случайное горизонтальное отражение изображения;
- случайный поворот изображения на угол до $\pm 10°$.

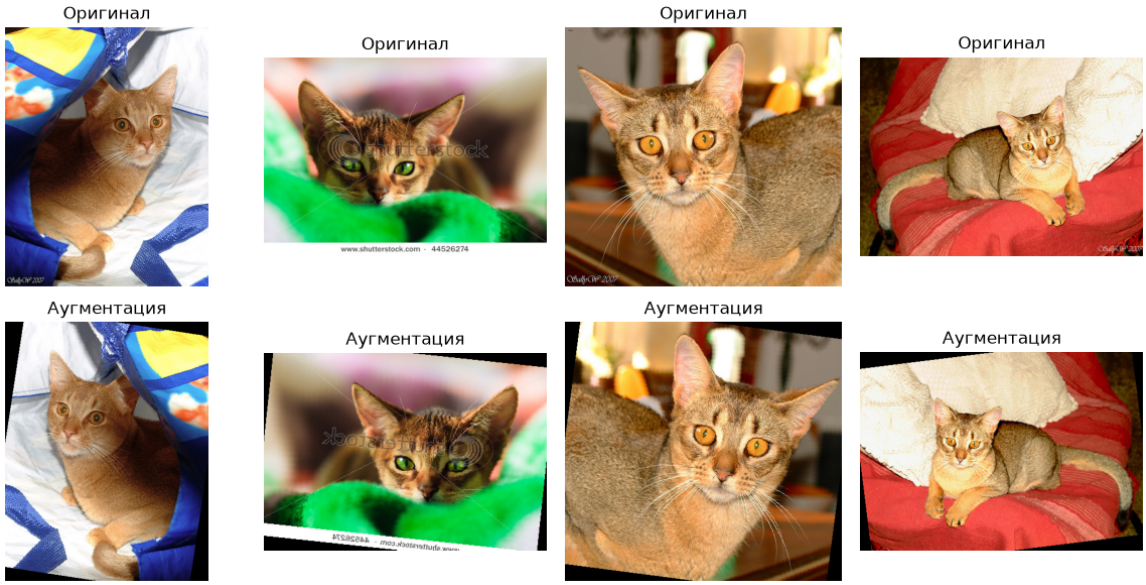


Рисунок 26 — Примеры аугментации изображений

2.3.2 Разделение датасета

Основной датасет разделяется на тренировочную и валидационную выборки в соотношении 80/20. В результате получены (рисунок 27):

- тренировочная выборка — 2944 изображения;
- валидационная выборка — 736 изображений;
- тестовая выборка — 3669 изображений.

```
Train: 2944
Validation: 736
Test: 3669
```

Рисунок 27 — Размеры датасетов после разделения

2.3.3 Загрузка данных

Для пакетной передачи данных используются объекты `DataLoader`. Размер пакета составляет 32 изображения. Для тренировочной выборки используется перемешивание данных. Итоговые параметры получившихся пакетов данных на рисунке 28.

```
Images:
        batch_size=32
        color_channels=3
        width=224
        height=224
Labels:
        batch_size=32
```

Рисунок 28 — Параметры пакетов данных

2.4 Feature Extraction

2.4.1 Настройка модели

Для первого подхода нужно у существующей модели (натренированной на ImageNet) заморозить все слои, чтобы их нельзя было переобучить, и заменить итоговый классификатор на подходящий для выбранного датасета на 37 класссов (рисунок 29).

```
weights = ResNet18_Weights.DEFAULT
model = models.resnet18(weights=weights)

# Замораживаем все параметры предобученной сети
for param in model.parameters():
    param.requires_grad = False

# Заменяем последний классификатор на 37 классов
model.fc = nn.Linear(model.fc.in_features, len(class_names))

# Переносим модель на GPU
model = model.to(device)
```

Рисунок 29 — Настройка модели (Feature Extraction)

2.4.2 Обучение модели

Обучение модели проводилось в течение 10 эпох с использованием функции потерь `CrossEntropyLoss` (сравнивает предсказанные моделью вероятности классов с правильной меткой класса) и оптимизатора `Adam` (автоматически адаптирует шаг обучения для каждого параметра). Скорость обучения (насколько сильно модель изменяет свои веса после одной ошибки) была установлена равной 0,001.

В процессе обучения после каждой эпохи рассчитывалась точность на тренировочной и валидационной выборках (рисунок 30) и величина функции

потерь (рисунок 31). Для дальнейшей оценки сохранялись веса модели, показавшей наилучший результат на валидационной выборке. Наилучшее значение точности на валидационной выборке составило $87,36\%$ и было получено на восьмой эпохе.

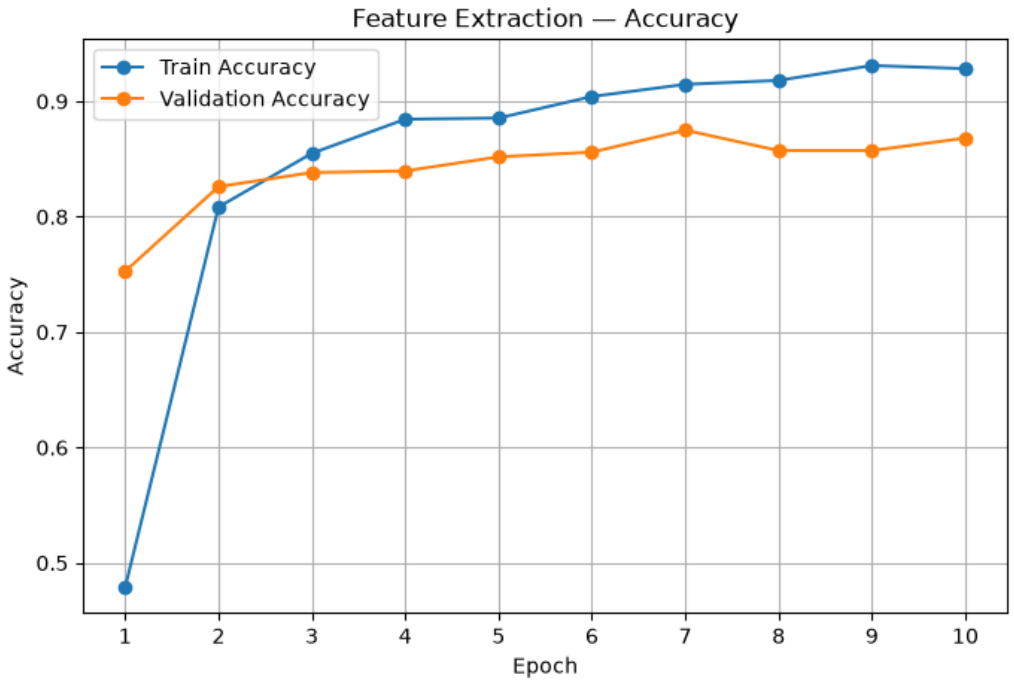


Рисунок 30 — Изменение Accuracy в процессе обучения Feature Extraction

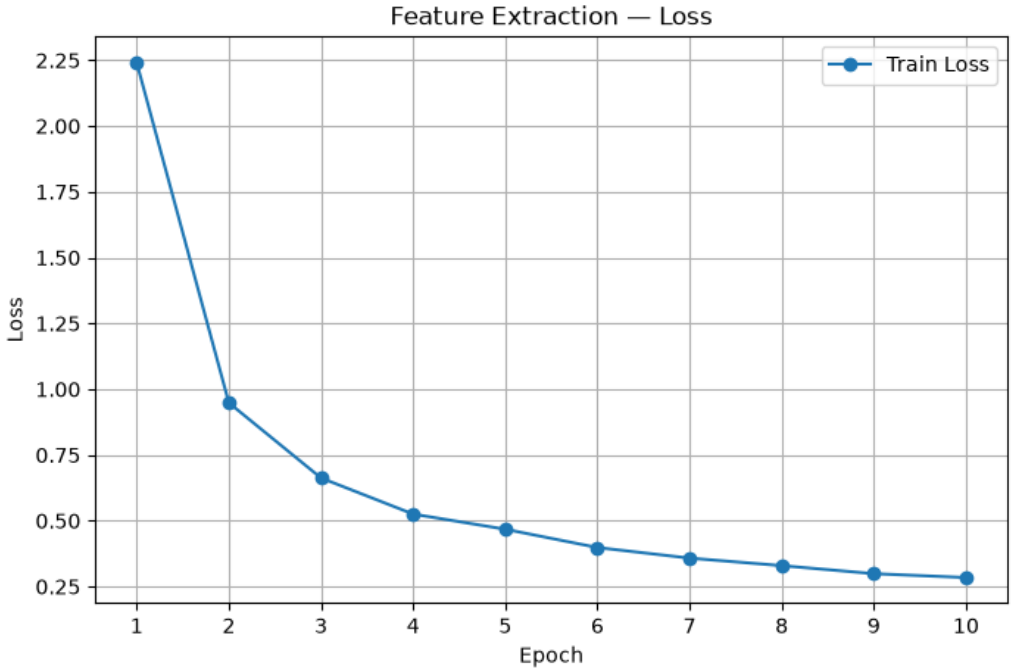


Рисунок 31 — Изменение функции потерь при Feature Extraction

2.4.3 Оценка модели

Результаты оценки модели Feature Extraction на тестовой выборке представлены в таблице 1. Матрица ошибок представлена на рисунке 32.

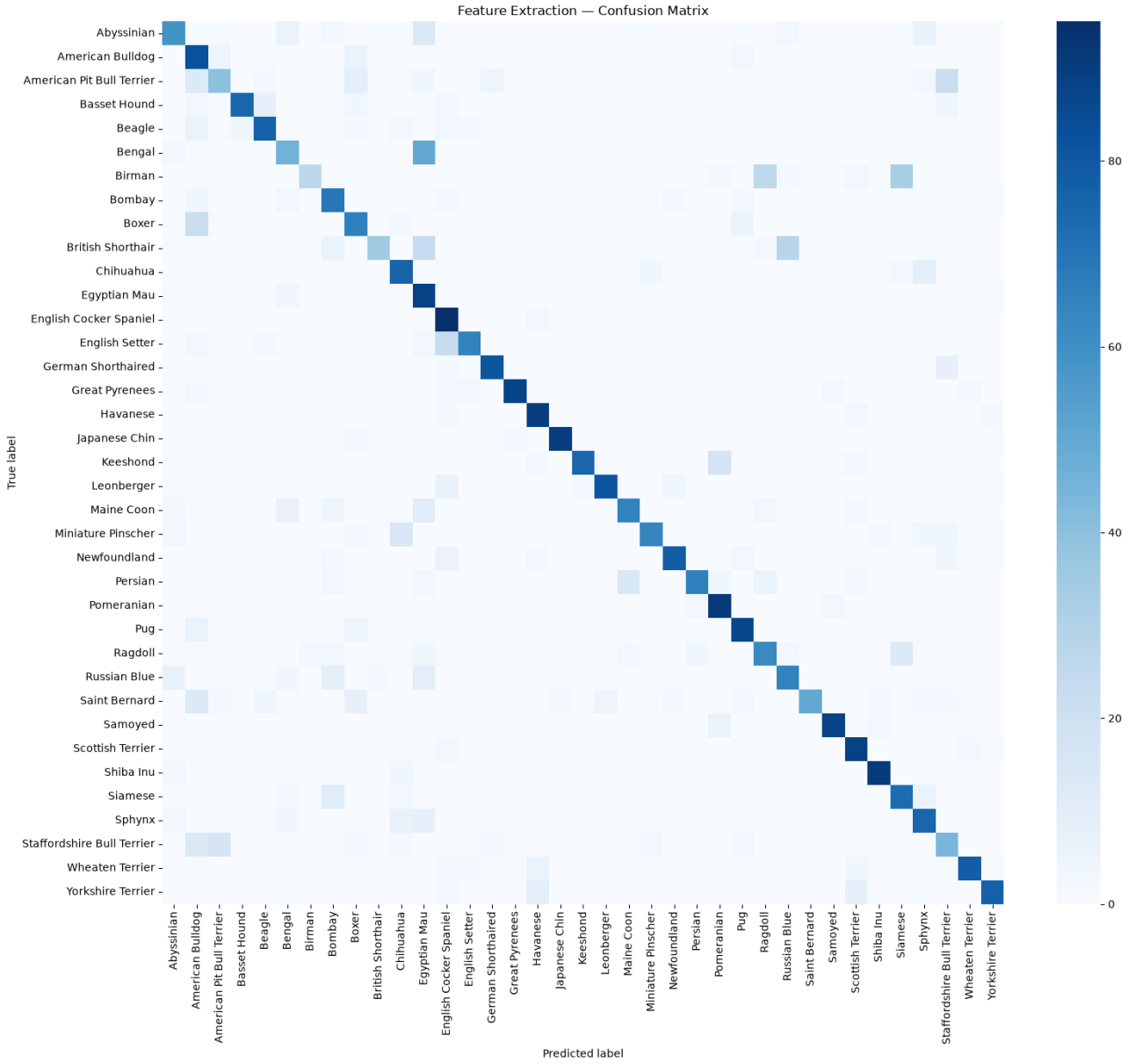


Рисунок 32 — Матрица ошибок модели Feature Extraction

Таблица 1 — Результаты Feature Extraction на тестовой выборке

Метрика	Значение
Accuracy	0.7225
Precision	0.7595
Recall	0.7222
F1-score	0.7210

Полученные результаты показывают, что модель правильно классифицирует большую часть изображений тестовой выборки. Значение Precision немного выше Recall, что указывает на несколько более высокую точность предсказаний отдельных классов по сравнению с полнотой их распознавания.

2.5 Fine-tuning

2.5.1 Настройка модели

Для второго подхода в отличие от Feature Extraction, часть слоёв модели размораживается для их дальнейшего дообучения на выбранном датасете. В работе размораживается последний остаточный блок `layer4`, а остальные слои остаются замороженными. Итоговый классификатор заменяется на новый слой с 37 выходами, соответствующими классам датасета (рисунок 33).

```
weights = ResNet18_Weights.DEFAULT
model_ft = models.resnet18(weights=weights)

# Замораживаем всю предобученную сеть
for param in model_ft.parameters():
param.requires_grad = False

# Размораживаем последний residual block
for param in model_ft.layer4.parameters():
param.requires_grad = True

# Заменяем классификатор на 37 классов
model_ft.fc = nn.Linear(
model_ft.fc.in_features,
len(class_names)
)

# Переносим модель на GPU
model_ft = model_ft.to(device)
```

Рисунок 33 — Настройка модели (Fine-tuning)

2.5.2 Обучение модели

Обучение модели проводилось в течение 10 эпох с использованием функции потерь `CrossEntropyLoss` и оптимизатора `Adam`. Скорость обучения была установлена равной 0,0001. Меньшая скорость обучения по сравнению с Feature Extraction используется для более осторожного изменения уже предобученных весов последнего блока сети.

В процессе обучения после каждой эпохи рассчитывалась точность на тренировочной и валидационной выборках (рисунок 34), а также величина функции потерь (рисунок 35). Для дальнейшей оценки сохранялись веса модели, показавшей наилучший результат на валидационной выборке. Наилуч-

шее значение точности на валидационной выборке составило $90,62\%$ и было получено на девятой эпохе.

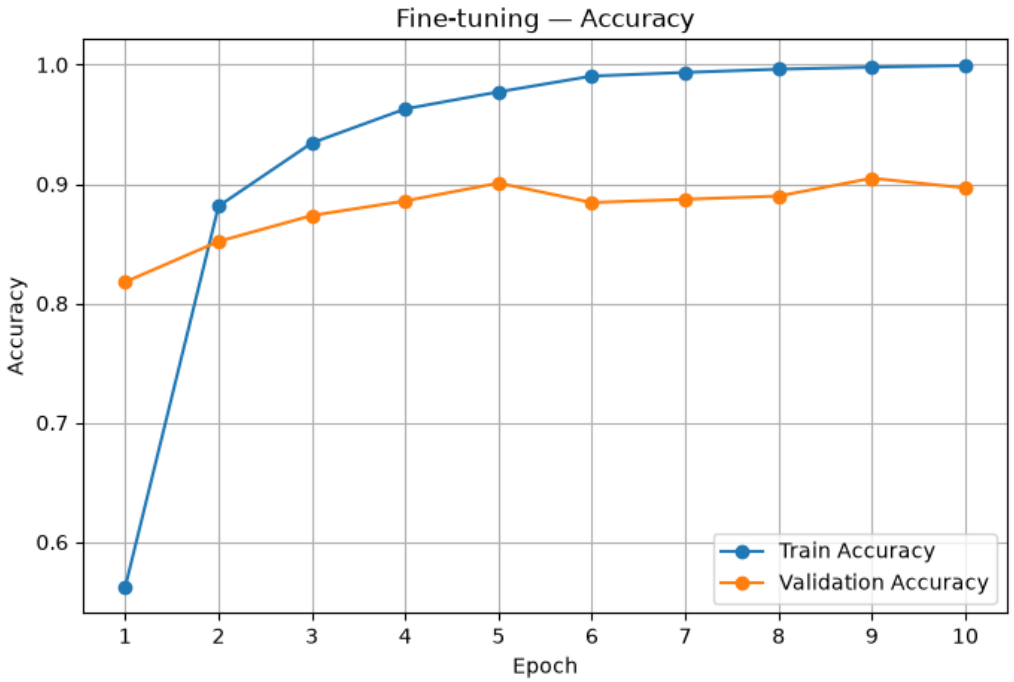


Рисунок 34 — Изменение Accuracy в процессе обучения Fine-tuning

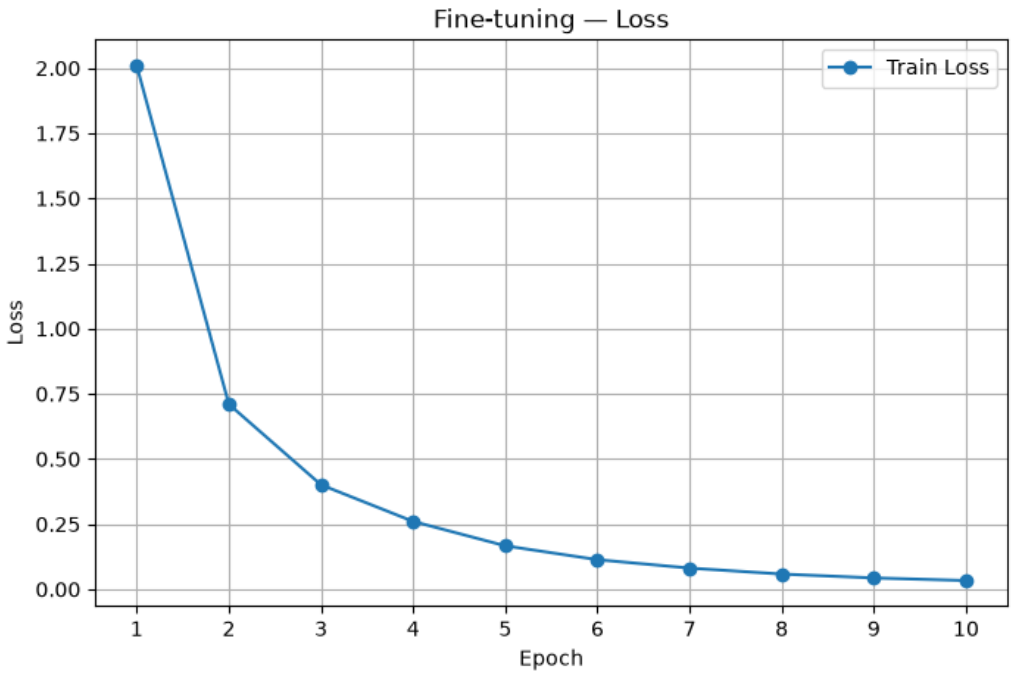


Рисунок 35 — Изменение функции потерь в процессе обучения Fine-tuning

2.5.3 Оценка модели

Результаты оценки модели Fine-tuning на тестовой выборке представлены в таблице 2. Матрица ошибок представлена на рисунке 36.

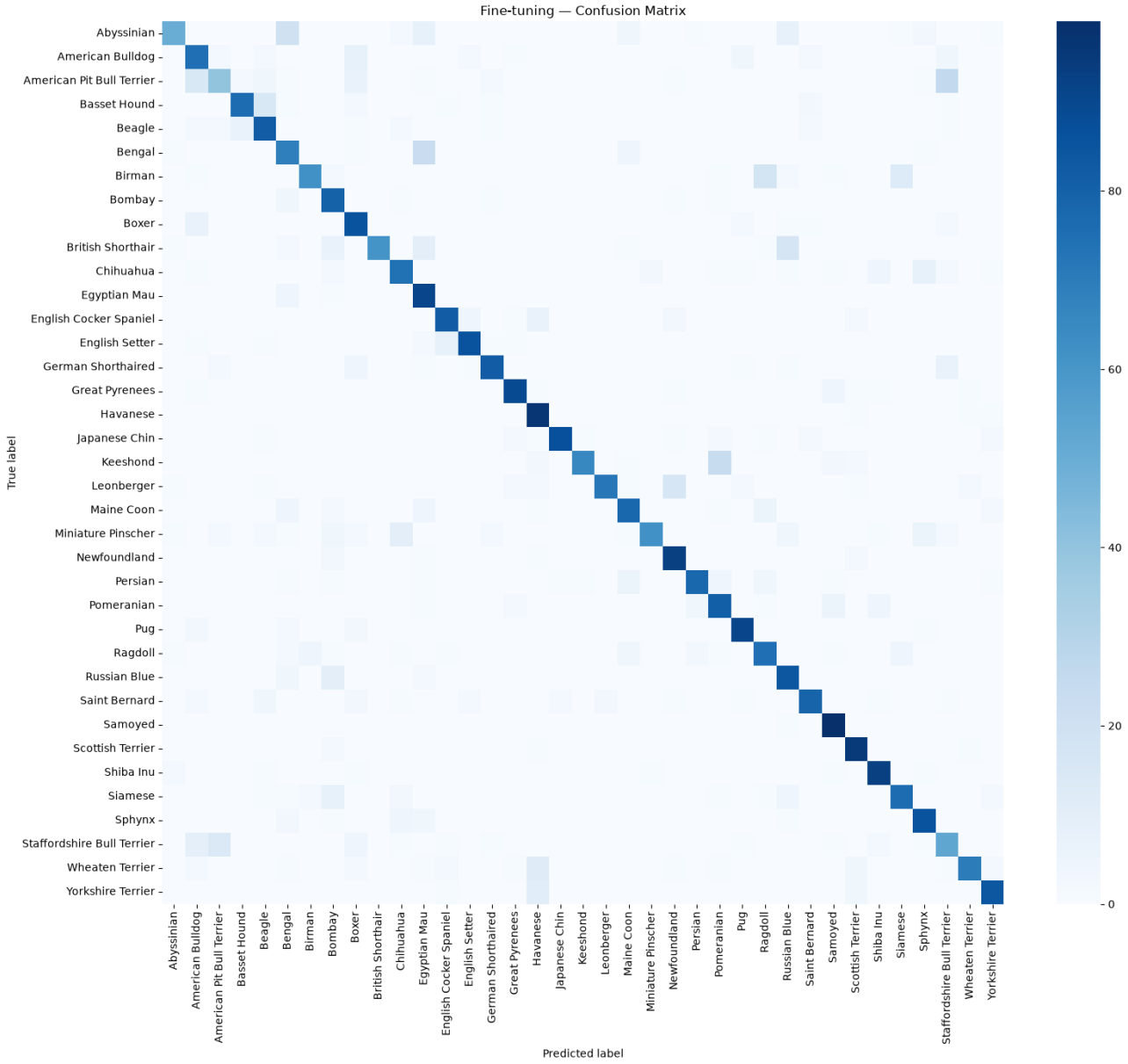


Рисунок 36 — Матрица ошибок модели Fine-tuning

Таблица 2 — Результаты Fine-tuning на тестовой выборке

Метрика	Значение
Accuracy	0.7860
Precision	0.8044
Recall	0.7859
F1-score	0.7842

Полученные результаты показывают, что после дообучения последнего блока модели удалось получить достаточно высокие значения основных метрик классификации. Значение Precision составило 80, 44%, а Recall — 78, 59%.

2.6 Сравнение

Сравнение метрик Feature Extraction и Fine-tuning на тестовой выборке представлено на рисунке 37 и в таблице 3.

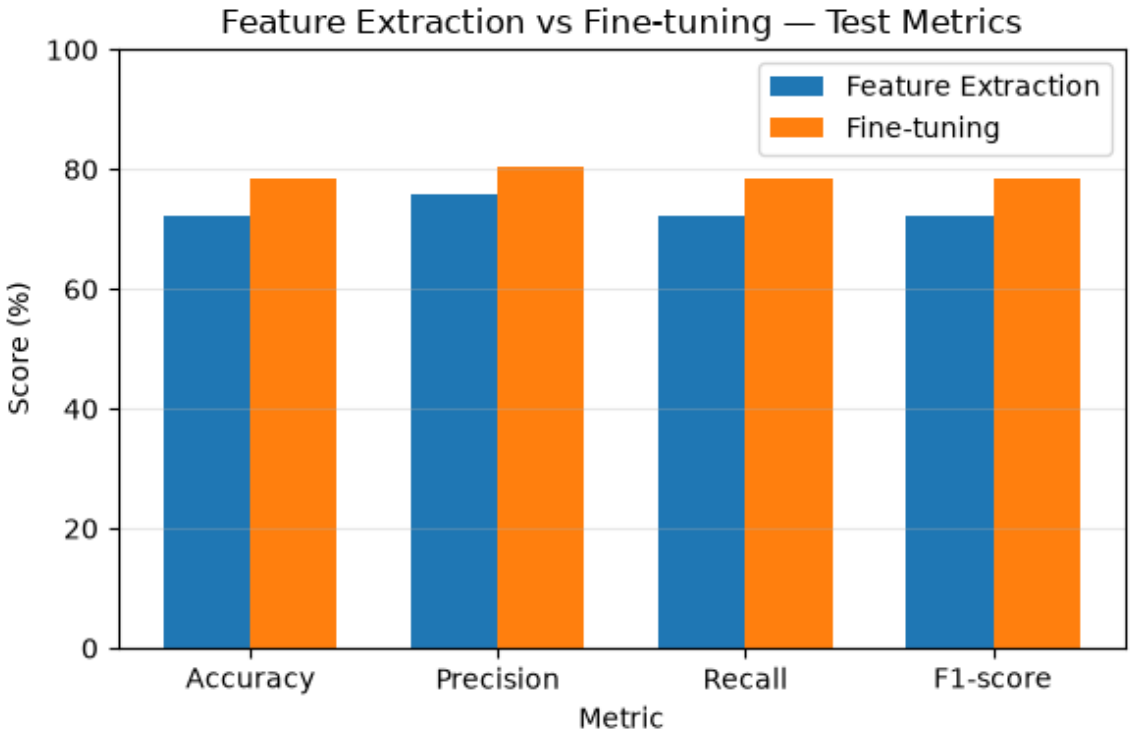


Рисунок 37 — Сравнение метрик Feature Extraction и Fine-tuning

Таблица 3 — Итоговое сравнение результатов на тестовой выборке

Метрика	Feature Extraction	Fine-tuning	Изменение
Accuracy	72,25%	78,60%	+6,35 п.п.
Precision	75,95%	80,44%	+4,49 п.п.
Recall	72,22%	78,59%	+6,37 п.п.
F1-score	72,10%	78,42%	+6,32 п.п.

По результатам работы было установлено, что переобучение части базовой модели увеличило все метрики в среднем на 6%: количество правильных предсказаний, их точность и полноту.

Таким образом, Fine-tuning оказался более эффективным подходом для рассматриваемой задачи. Дообучение верхнего блока предобученной сети позволило адаптировать сформированные признаки под особенности изображений пород кошек и собак, что привело к увеличению качества классификации. Поэтому для данного датасета применение Fine-tuning является целесообразным.

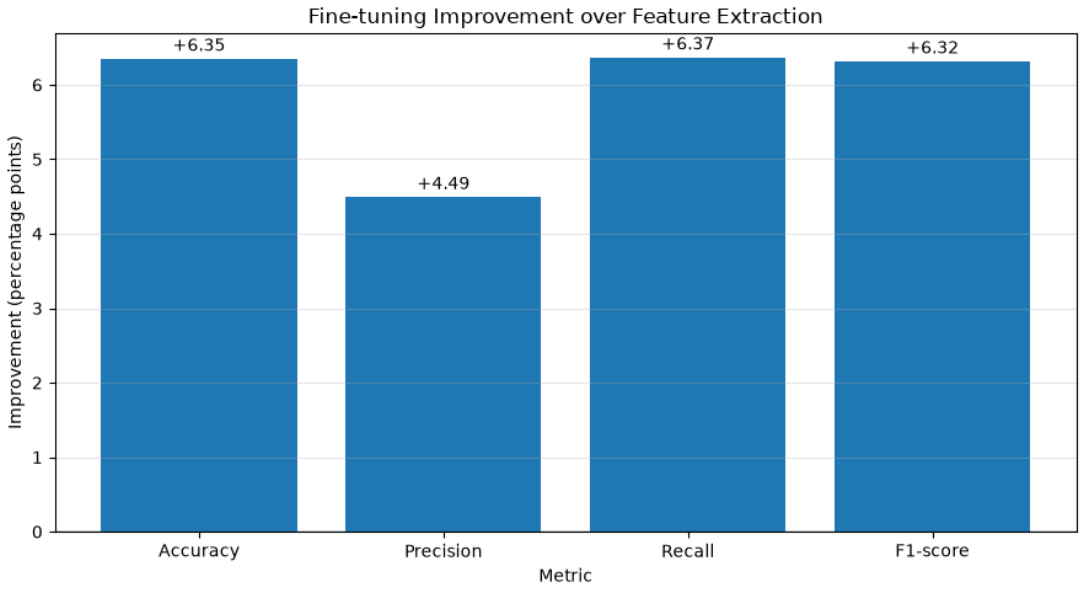


Рисунок 38 — Изменение метрик при переходе к Fine-tuning

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы были изучены основные методы предварительной обработки данных, классификации и регрессии с использованием библиотеки `sklearn`. Были рассмотрены логистическая регрессия, наивный байесовский классификатор, SVM и методы регрессии.

В ходе работы над дополнительной частью были реализованы стратегии Feature Extraction и Fine-tuning для классификации изображений датасета Oxford-IIIT Pet с использованием предобученной архитектуры ResNet18.

В результате Feature Extraction была получена Accuracy $72,25\%$, тогда как применение Fine-tuning позволило повысить её до $78,60\%$. Также увеличились значения Precision, Recall и F1-score. Таким образом, Fine-tuning показал более высокие результаты по всем рассмотренным метрикам.

По результатам работы было установлено, что дообучение верхних слоёв предобученной модели позволяет улучшить качество классификации. Поэтому для рассматриваемого датасета применение Fine-tuning является целесообразным.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Исходный код работы [Электронный ресурс]. — URL: https://github.com/OneTwoZzzPlus/modern-ai-methods/ (дата обращения: 04.10.2026).

2. The Oxford-IIIT Pet Dataset [Электронный ресурс]. — URL: https://www.kaggle.com/datasets/tanlikesmath/the-oxfordiiit-pet-dataset (дата обращения: 04.10.2026).